

# HAMi Multi-Agent Sistem

GPU Sanallaştırma ile Agent Sınıflandırma

---

## Özet

---

|                         |                                                                                                 |
|-------------------------|-------------------------------------------------------------------------------------------------|
| <b>Platform:</b>        | Kubernetes (kubeadm v1.29.15)                                                                   |
| <b>GPU:</b>             | NVIDIA GeForce GTX 1650 (4GB VRAM)                                                              |
| <b>İşletim Sistemi:</b> | Ubuntu 24.04 LTS                                                                                |
| <b>HAMi:</b>            | v2.8.0                                                                                          |
| <b>GitHub:</b>          | <a href="https://github.com/ysf0kaya/hami-multi-agent">github.com/ysf0kaya/hami-multi-agent</a> |
| <b>Tarih:</b>           | Mayıs 2026                                                                                      |

# İçindekiler

|          |                                                |           |
|----------|------------------------------------------------|-----------|
| <b>1</b> | <b>Projeye Genel Bakış</b>                     | <b>2</b>  |
| 1.1      | Danışman Önerisi . . . . .                     | 2         |
| 1.2      | Sistem Bileşenleri . . . . .                   | 2         |
| <b>2</b> | <b>Agent Kategorileri ve Model Seçimi</b>      | <b>2</b>  |
| 2.1      | Kategori Tanımları . . . . .                   | 2         |
| 2.2      | HuggingFace'den Seçilen Modeller . . . . .     | 3         |
| 2.2.1    | GPU_YOGUN — LLM Modelleri . . . . .            | 3         |
| 2.2.2    | CPU_YOGUN — NLP Modelleri . . . . .            | 3         |
| 2.2.3    | IO_YOGUN — Vision Modelleri . . . . .          | 4         |
| <b>3</b> | <b>Veri Toplama Sistemi</b>                    | <b>4</b>  |
| 3.1      | Metrik Toplama Stratejisi . . . . .            | 4         |
| 3.2      | Toplanan Metrikler . . . . .                   | 5         |
| 3.3      | Görev Tipleri . . . . .                        | 5         |
| 3.4      | Dataset İstatistikleri . . . . .               | 5         |
| 3.5      | Kategori Profilleri . . . . .                  | 6         |
| <b>4</b> | <b>Makine Öğrenmesi Modeli</b>                 | <b>6</b>  |
| 4.1      | Model Seçimi: Random Forest . . . . .          | 6         |
| 4.2      | Veri Ön İşleme . . . . .                       | 6         |
| 4.3      | Train/Val/Test Bölünmesi . . . . .             | 6         |
| 4.4      | Model Hiperparametreleri . . . . .             | 6         |
| 4.5      | Model Performansı . . . . .                    | 7         |
| 4.5.1    | Sınıf Bazında Performans (Test Seti) . . . . . | 7         |
| 4.5.2    | Confusion Matrix . . . . .                     | 7         |
| 4.6      | Feature Importance . . . . .                   | 8         |
| <b>5</b> | <b>Tahmin Sistemi</b>                          | <b>8</b>  |
| 5.1      | predict.py — Yeni Agent Tahmini . . . . .      | 8         |
| 5.2      | Test Sonuçları . . . . .                       | 9         |
| <b>6</b> | <b>HAMi GPU Sanallaştırma</b>                  | <b>9</b>  |
| 6.1      | VRAM İzolasyonu . . . . .                      | 9         |
| 6.2      | HAMi WebUI Gözlemi . . . . .                   | 9         |
| <b>7</b> | <b>Proje Yapısı</b>                            | <b>9</b>  |
| <b>8</b> | <b>Karşılaşılan Zorluklar ve Çözümler</b>      | <b>10</b> |
| <b>9</b> | <b>Sonuç ve Gelecek Çalışmalar</b>             | <b>10</b> |
| 9.1      | Elde Edilen Sonuçlar . . . . .                 | 10        |
| 9.2      | Gelecek Çalışmalar . . . . .                   | 11        |
| <b>A</b> | <b>Ek: Örnek Dataset Satırları</b>             | <b>11</b> |

## 1. Projeye Genel Bakış

Bu proje, Kubernetes ortamında HAMi (Heterogeneous AI Computing Virtualization Middleware) kullanarak GPU kaynaklarını birden fazla yapay zeka ajanı arasında paylaştıran ve bu ajanları HAMi metriklerine dayalı olarak otomatik sınıflandıran bir sistem prototipini sunmaktadır.

### 1.1. Danışman Önerisi

Proje danışmanının yönlendirmesi doğrultusunda sistem üç temel işlevi kapsamaktadır:

1. **Veri Toplama:** HAMi üzerinden çalışan agent'lerden GPU, CPU ve bellek metriklerinin toplanması
2. **Model Eğitimi:** Toplanan verilerle agent kategorilerini öğrenen bir makine öğrenmesi modelinin eğitilmesi
3. **Sınıflandırma:** Yeni bir agent Kubernetes'e deploy edildiğinde, HAMi metriklerine bakılarak otomatik kategori tahmini yapılması

### 1.2. Sistem Bileşenleri

Tablo 1: Sistem Bileşenleri

| Bileşen                  | Versiyon              | Görev                     |
|--------------------------|-----------------------|---------------------------|
| Kubernetes               | v1.29.15              | Container orkestrasyon    |
| Docker                   | CE                    | Container runtime         |
| HAMi                     | v2.8.0                | GPU sanallaştırma         |
| HAMi WebUI               | latest                | GPU izleme arayüzü        |
| Prometheus               | kube-prometheus-stack | Metrik toplama            |
| Python                   | 3.12                  | Agent ve model geliştirme |
| PyTorch                  | 2.3.0                 | GPU computing framework   |
| HuggingFace Transformers | 4.41.0                | Hazır AI modelleri        |
| scikit-learn             | 1.4.2                 | Sınıflandırma modeli      |

## 2. Agent Kategorileri ve Model Seçimi

### 2.1. Kategori Tanımları

Danışman önerisi doğrultusunda agent'lar üç kategoride sınıflandırılmıştır:

Tablo 2: Agent Kategorileri

| Kategori         | Açıklama                | Beklenen Profil              |
|------------------|-------------------------|------------------------------|
| <b>GPU_YOGUN</b> | LLM inference ağırlıklı | Yüksek VRAM, düşük CPU       |
| <b>CPU_YOGUN</b> | NLP/text classification | Düşük VRAM, yüksek CPU       |
| <b>IO_YOGUN</b>  | Görüntü işleme          | Orta VRAM, yüksek throughput |

## 2.2. HuggingFace'den Seçilen Modeller

Her kategoriden birden fazla model seçilerek dataset çeşitliliği sağlanmıştır.

### 2.2.1. GPU\_YOGUN — LLM Modelleri

Tablo 3: GPU Yoğunluklu Agent Modelleri

| Agent ID      | Model                  | Parametre | VRAM Kotası |
|---------------|------------------------|-----------|-------------|
| llm-distilgpt | distilgpt2             | 82M       | 1024 MB     |
| llm-lamini    | MBZUAI/LaMini-GPT-124M | 124M      | 1024 MB     |
| llm-opt       | facebook/opt-125m      | 125M      | 1024 MB     |

**Not:** Başlangıçta Qwen/Qwen1.5-0.5B ve TinyLlama/TinyLlama-1.1B modelleri de seçilmişti; ancak bu modeller GTX 1650'nin 4GB VRAM kapasitesini aşarak sistem donmalarına yol açtığından listeden çıkarılmıştır.

### 2.2.2. CPU\_YOGUN — NLP Modelleri

Tablo 4: CPU Yoğunluklu Agent Modelleri

| Agent ID       | Model                      | Parametre | VRAM Kotası | Ko- |
|----------------|----------------------------|-----------|-------------|-----|
| nlp-bert-tiny  | prajjwal1/bert-tiny        | 4.4M      | 256 MB      |     |
| nlp-bert-base  | bert-base-uncased          | 110M      | 768 MB      |     |
| nlp-distilbert | distilbert-base-uncased    | 66M       | 768 MB      |     |
| nlp-roberta    | cardiffnlp/twitter-roberta | 125M      | 768 MB      |     |
| nlp-albert     | albert-base-v2             | 12M       | 768 MB      |     |

### 2.2.3. IO\_YOGUN — Vision Modelleri

Tablo 5: IO Yoğunluklu Agent Modelleri

| Agent ID            | Model                   | Görev          | VRAM<br>tası | Ko- |
|---------------------|-------------------------|----------------|--------------|-----|
| vision-resnet18     | microsoft/resnet-18     | Görüntü sınıf. | 768 MB       |     |
| vision-resnet50     | microsoft/resnet-50     | Görüntü sınıf. | 768 MB       |     |
| vision-mobilenet    | google/mobilenet_v2     | Görüntü sınıf. | 768 MB       |     |
| vision-vit          | google/vit-base-patch16 | Görüntü sınıf. | 768 MB       |     |
| vision-efficientnet | google/mobilenet_v2*    | Görüntü sınıf. | 768 MB       |     |

\*EfficientNet HuggingFace AutoFeatureExtractor ile uyumsuz olduğu için MobileNet ile değiştirilmiştir.

## 3. Veri Toplama Sistemi

### 3.1. Metrik Toplama Stratejisi

Danışman önerisi doğrultusunda **ortalama kullanım** temelli veri toplama yapılmıştır. Her agent için şu akış uygulanmıştır:

1. Model HuggingFace'den indirilir ve GPU'ya yüklenir
2. **32 örnekten oluşan batch**, 5 kez tekrar çalıştırılır
3. Her batch sonrasında anlık metrikler kaydedilir
4. 5 batch tamamlandıktan sonra ortalama hesaplanır ve CSV'ye yazılır

### 3.2. Toplanan Metrikler

Tablo 6: Dataset Özellikleri (Feature'lar)

| Özellik                 | Kaynak       | Açıklama                       |
|-------------------------|--------------|--------------------------------|
| avg_gpu_mem_mb          | HAMi/PyTorch | Ortalama VRAM kullanımı (MB)   |
| max_gpu_mem_mb          | HAMi/PyTorch | Maksimum VRAM kullanımı (MB)   |
| avg_gpu_compute_percent | HAMi/PyTorch | Ortalama GPU compute oranı (%) |
| avg_cpu_percent         | psutil       | Ortalama CPU kullanımı (%)     |
| max_cpu_percent         | psutil       | Maksimum CPU kullanımı (%)     |
| avg_ram_gb              | psutil       | Ortalama RAM kullanımı (GB)    |
| avg_batch_duration_s    | Hesaplanan   | Batch başına ortalama süre (s) |
| throughput_per_s        | Hesaplanan   | Saniyedeki batch sayısı        |

### 3.3. Görev Tipleri

Tablo 7: Kategori Bazında Görev Tipleri

| Kategori  | Görev                      | Batch İçeriği                   |
|-----------|----------------------------|---------------------------------|
| GPU_YOGUN | Text generation            | 32 prompt, max 50 token üretimi |
| CPU_YOGUN | Fill-mask / Classification | 32 metin, mask doldurma         |
| IO_YOGUN  | Image classification       | 32 rastgele 224×224 görüntü     |

### 3.4. Dataset İstatistikleri

Toplam **36 satır** veri toplanmıştır:

Tablo 8: Dataset Dağılımı

| Kategori      | Satır Sayısı | Model Sayısı | Oran (%)   |
|---------------|--------------|--------------|------------|
| CPU_YOGUN     | 22           | 5            | 61.1       |
| GPU_YOGUN     | 9            | 3            | 25.0       |
| IO_YOGUN      | 5            | 5            | 13.9       |
| <b>Toplam</b> | <b>36</b>    | <b>13</b>    | <b>100</b> |

### 3.5. Kategori Profilleri

Toplanan verilerden elde edilen ortalama metrikler kategoriler arasında belirgin farklılıklar göstermektedir:

Tablo 9: Kategori Bazında Ortalama Metrikler

| Kategori  | Avg GPU Mem (MB) | Avg CPU (%) | Throughput (b/s) | Batch Süresi (s) |
|-----------|------------------|-------------|------------------|------------------|
| GPU_YOGUN | ~457             | ~9          | ~1.5             | ~0.67            |
| CPU_YOGUN | ~228             | ~49         | ~7.7             | ~0.14            |
| IO_YOGUN  | ~149             | ~8          | ~12.4            | ~0.09            |

## 4. Makine Öğrenmesi Modeli

### 4.1. Model Seçimi: Random Forest

Sınıflandırma için **Random Forest** algoritması seçilmiştir. Seçim gerekçeleri:

- Az veri ile iyi performans göstermesi (36 satır dataset)
- Yorumlanabilir feature importance çıktısı
- Dengesiz sınıf dağılımına karşı `class_weight="balanced"` desteği
- Normalleştirilmiş veriye ihtiyaç duymaması (StandardScaler uygulandı)

### 4.2. Veri Ön İşleme

1. **Feature seçimi:** 8 sayısal özellik seçildi
2. **Label encoding:** CPU\_YOGUN=0, GPU\_YOGUN=1, IO\_YOGUN=2
3. **StandardScaler:** Özellikler normalize edildi
4. **Train/Val/Test bölünmesi:** %64 / %16 / %20 (stratified)

### 4.3. Train/Val/Test Bölünmesi

Tablo 10: Veri Bölünmesi

| Küme       | Satır Sayısı | Oran | Yöntem           |
|------------|--------------|------|------------------|
| Train      | 22           | %64  | Stratified split |
| Validation | 6            | %16  | Stratified split |
| Test       | 8            | %20  | Stratified split |

### 4.4. Model Hiperparametreleri

```

1 model = RandomForestClassifier(
2     n_estimators=100,      # 100 karar agaci
3     max_depth=10,         # Maksimum derinlik
4     random_state=42,      # Tekrarlanabilirlik
5     class_weight="balanced" # Dengesiz sinif icin
6 )

```

Listing 1: Random Forest Hiperparametreleri

## 4.5. Model Performansı

Tablo 11: Model Performans Metrikleri

| Metrik      | Train | Validation | Test          |
|-------------|-------|------------|---------------|
| Accuracy    | -     | 1.0000     | <b>0.8750</b> |
| Macro F1    | -     | -          | 0.64          |
| Weighted F1 | -     | -          | 0.82          |

### 4.5.1. Sınıf Bazında Performans (Test Seti)

Tablo 12: Classification Report

| Sınıf           | Precision | Recall | F1-Score    | Support  |
|-----------------|-----------|--------|-------------|----------|
| CPU_YOGUN       | 0.83      | 1.00   | 0.91        | 5        |
| GPU_YOGUN       | 1.00      | 1.00   | 1.00        | 2        |
| IO_YOGUN        | 0.00      | 0.00   | 0.00        | 1        |
| <b>Accuracy</b> |           |        | <b>0.88</b> | <b>8</b> |
| Macro avg       | 0.61      | 0.67   | 0.64        | 8        |
| Weighted avg    | 0.77      | 0.88   | 0.82        | 8        |

**IO\_YOGUN performansı:** Test setinde yalnızca 1 örnek bulunmakta ve bu örnek yanlış sınıflandırılmaktadır. Bu durum veri azlığından kaynaklanmakta olup daha fazla vision agent çalıştırılarak iyileştirilebilir.

### 4.5.2. Confusion Matrix

$$\begin{pmatrix} 5 & 0 & 0 \\ 0 & 2 & 0 \\ 1 & 0 & 0 \end{pmatrix}$$

Satırlar: Gerçek değerler (CPU\_YOGUN, GPU\_YOGUN, IO\_YOGUN)

Sütunlar: Tahmin edilen değerler



## 4.6. Feature Importance

Tablo 13: Özellik Önem Sıralaması

| Özellik                 | Önem Skoru | Yorum                    |
|-------------------------|------------|--------------------------|
| throughput_per_s        | 0.2203     | En ayırt edici özellik   |
| avg_batch_duration_s    | 0.1889     | Görev süresi belirleyici |
| avg_cpu_percent         | 0.1599     | CPU yükü önemli          |
| avg_ram_gb              | 0.1568     | RAM destek özelliği      |
| max_cpu_percent         | 0.0891     | Tepe CPU değeri          |
| max_gpu_mem_mb          | 0.0718     | GPU bellek destek        |
| avg_gpu_compute_percent | 0.0653     | GPU compute oranı        |
| avg_gpu_mem_mb          | 0.0477     | En az belirleyici        |

**Önemli bulgu:** En belirleyici özelliğin GPU belleği değil, *throughput* (işlem hızı) olması dikkat çekicidir. Bu, kategoriler arası farkın VRAM kullanımından ziyade görev hızından kaynaklandığını göstermektedir.

## 5. Tahmin Sistemi

### 5.1. predict.py — Yeni Agent Tahmini

Eğitilmiş model yeni bir agent'ın HAMi metriklerine bakarak kategorisini tahmin eder:

```

1 from model.predict import predict
2
3 metrics = {
4     "avg_gpu_mem_mb": 400.0,
5     "max_gpu_mem_mb": 400.0,
6     "avg_gpu_compute_percent": 40.0,
7     "avg_cpu_percent": 8.0,
8     "max_cpu_percent": 20.0,
9     "avg_ram_gb": 5.7,
10    "avg_batch_duration_s": 0.6,
11    "throughput_per_s": 1.5,
12 }
13
14 result = predict(metrics)
15 # {'category': 'GPU_YOGUN', 'confidence': 0.88,
16 #   'probabilities': {'CPU_YOGUN': 0.06,
17 #                     'GPU_YOGUN': 0.88,
18 #                     'IO_YOGUN': 0.06}}
```

Listing 2: predict.py kullanımı

## 5.2. Test Sonuçları

Tablo 14: Tahmin Test Sonuçları

| Test Senaryosu    | Beklenen  | Tahmin    | Güven |
|-------------------|-----------|-----------|-------|
| Yeni LLM Agent    | GPU_YOGUN | GPU_YOGUN | %88   |
| Yeni NLP Agent    | CPU_YOGUN | CPU_YOGUN | %100  |
| Yeni Vision Agent | IO_YOGUN  | IO_YOGUN  | %71   |

Tüm test senaryoları doğru sınıflandırılmıştır.

## 6. HAMi GPU Sanallaştırma

### 6.1. VRAM İzolasyonu

HAMi'nin temel katkısı, tek fiziksel GPU'yu birden fazla container arasında güvenli biçimde paylaşmasıdır:

```

1 resources:
2   limits:
3     nvidia.com/gpu: 1
4     nvidia.com/gpumem: 1024 # 1GB VRAM kotasi

```

Listing 3: HAMi VRAM Kota Tanımı (K8s Manifest)

### 6.2. HAMi WebUI Gözlemi

Sistem çalışır haldeyken HAMi WebUI'de şu bilgiler görüntülenmiştir:

- **GPU:** NVIDIA GeForce GTX 1650 — Healthy
- **Compute Alloc:** %300 (3 agent × %100 compute kotası)
- **Memory Alloc:** 3/4 GiB
- **Task List:** 3 agent aynı anda Running durumunda

## 7. Proje Yapısı

```

1 hami-multi-agent/
2 |-- agents/
3 |   |-- base_agent.py           # Ortak temel sınıf
4 |   |-- llm_agent.py            # GPU_YOGUN: LLM modelleri
5 |   |-- nlp_agent.py            # CPU_YOGUN: BERT modelleri
6 |   |-- vision_agent.py         # IO_YOGUN: Vision modelleri
7 |   '-- audio_agent.py          # (KARMA: devre dışı)
8 |-- data/

```

```

9 | |-- collector.py           # Metrik toplama
10 | '-- processed/           # Islenenmis veri
11 |-- model/
12 | |-- train.py            # Model eğitimi
13 | |-- predict.py          # Tahmin
14 | '-- saved/              # Eğitilmiş model
15 |-- k8s/                 # 18 Kubernetes manifest
16 |-- docker/
17 | '-- Dockerfile.agent    # Tüm agentlar için
18 |-- scripts/
19 | |-- collect_data.sh      # Otomatik veri toplama
20 | '-- generate_manifests.sh # Manifest üretici
21 '-- requirements.txt

```

Listing 4: Proje Klasör Yapısı

## 8. Karşılaşılan Zorluklar ve Çözümler

Tablo 15: Karşılaşılan Problemler ve Çözümleri

| Problem                       | Sebebi                           | Çözüm                              |
|-------------------------------|----------------------------------|------------------------------------|
| Qwen/TinyLlama sistem donması | 4GB VRAM yetersiz                | Küçük modeller seçildi             |
| OOM hataları                  | VRAM kotası düşük                | YAML'da gpumem artırıldı           |
| wav2vec2 OOM                  | Batch işlem çok büyük            | Model listeden çıkarıldı           |
| EfficientNet hata             | AutoFeatureExtractor uyumsuzluğu | MobileNet ile değiştirildi         |
| Dataset kalıcılığı            | /tmp yeniden başlatmada silindi  | /home/ysf/hami-dataset kullanıldı  |
| Docker DNS hatası             | daemon.json'da DNS yok           | 8.8.8.8 eklendi                    |
| IO_YOGUN düşük F1             | Sadece 5 veri noktası            | Daha fazla vision agent öneriliyor |
| fill-mask truncation          | Pipeline uyumsuzluğu             | Parametre kaldırıldı               |
| GPT2 pad_token                | Pad token tanımsız               | eos_token_id atandı                |

## 9. Sonuç ve Gelecek Çalışmalar

### 9.1. Elde Edilen Sonuçlar

- HAMi ile tek GPU'da 3 farklı kategoride agent başarıyla çalıştırılmıştır
- 13 farklı HuggingFace modelinden 36 satır dataset toplanmıştır
- Random Forest modeli %87.5 test doğruluğu ile eğitilmiştir
- Yeni agent tahmini için çalışan bir sistem oluşturulmuştur

## 9.2. Gelecek Çalışmalar

1. **Monitoring Servisi:** Agent'ların aynı anda çalışırken Prometheus'tan sürekli metrik çeken ve otomatik sınıflandırma yapan servis
2. **IO\_YOGUN verisi artırma:** Daha fazla vision agent çalıştırılarak model performansı iyileştirilmesi
3. **Anomali tespiti:** Normal davranış profilinden sapan agent'ların tespiti
4. **Otomatik kaynak önerisi:** Sınıflandırma sonucuna göre HAMi VRAM kotasının otomatik ayarlanması
5. **REST API:** Coordinator servisi üzerinden HTTP ile görev gönderimi

## A. Ek: Örnek Dataset Satırları

Tablo 16: Dataset Örnek Satırları

| agent_id        | model_name | category  | avg_gpu_m | avg_cpu | throughput | batch_dur |
|-----------------|------------|-----------|-----------|---------|------------|-----------|
| nlp-bert-tiny   | bert-tiny  | CPU_YOGUN | 26.2 MB   | 58.9%   | 8.40/s     | 0.12s     |
| llm-distilgpt   | distilgpt2 | GPU_YOGUN | 343.3 MB  | 9.4%    | 1.83/s     | 0.55s     |
| vision-resnet18 | resnet-18  | IO_YOGUN  | 74.8 MB   | 9.5%    | 13.24/s    | 0.08s     |
| nlp-bert-base   | bert-base  | CPU_YOGUN | 447.7 MB  | 55.7%   | 5.75/s     | 0.17s     |
| vision-vit      | vit-base   | IO_YOGUN  | 374.2 MB  | 2.7%    | 4.50/s     | 0.22s     |